

→ Adjusts weights & biases to minimize function

Optimizer

→ Most imp technique to speed up Neural Networks

We use Gradient Descent optimizer in deep learning

$$W_n = W_0 - \eta \frac{dL}{dW_0}$$

if 100 epoch
→ 100 update

Batch GD → 500 rows → 1 update on whole data in each epoch
SGD → 500 rows → every update on every epoch
Mini Batch GD

Batch size = 100

↓
Sbatch → 100 x 5 = 500
↑
epochs

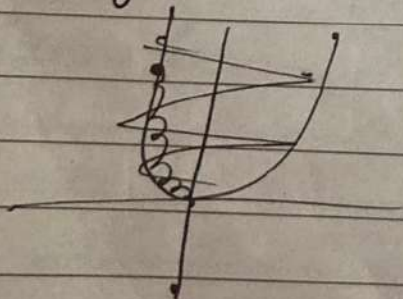
if 100 epoch
500 rows

$$500 \times 1000$$

Challenges

Learning rate

$$W_n = W_0 + \eta \frac{dL}{dW_0}$$

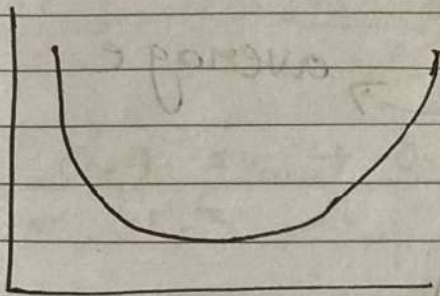


↓
if small, slow convergence
if large, may skip minima

2. Learning rate Scheduling

↓ need predefine η
when to in, when to dec
doesn't depend on dataset, so it's challenging

3

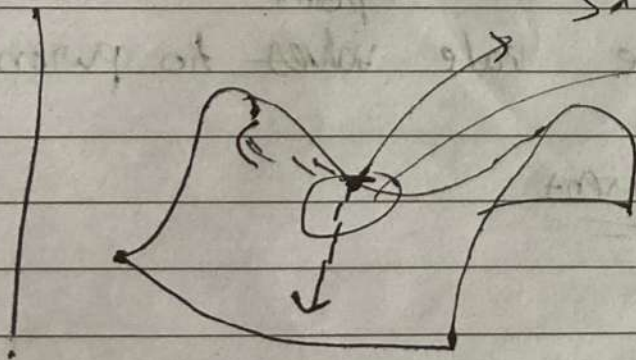


if there are more than 1 weights, we need to initialize different learning rate for them

4. Local minima

May not get global minima & get stuck in local minima

5. Saddle point



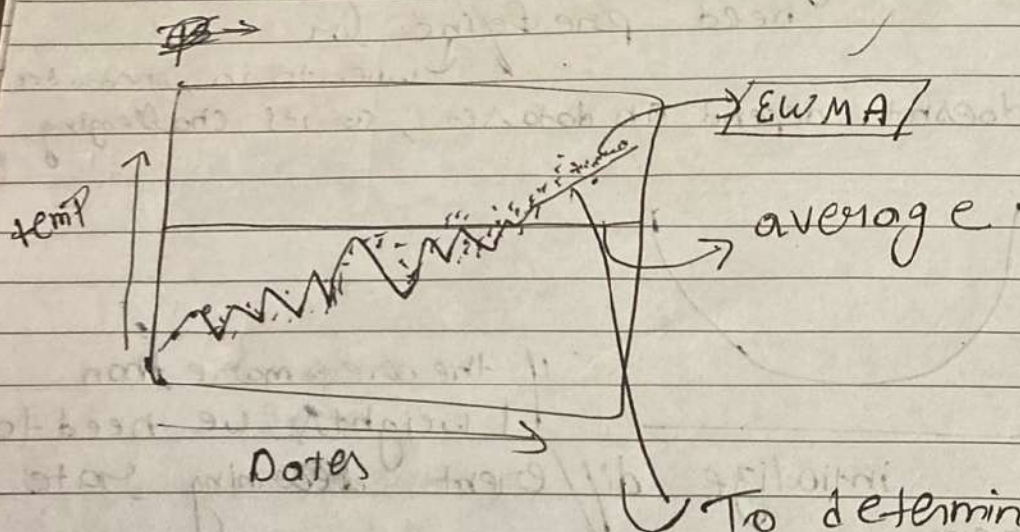
slope going up & down also

no slope change

$$\frac{dL}{dW} = 0$$

↓
vanishing gradient

Exponentially Moving Weighted Moving Average (EWMA)



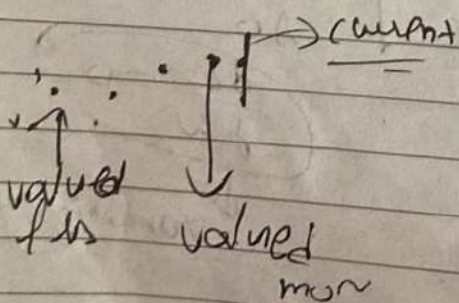
- Time Series
 - financial
 - Signal price
 - Deep learning
- uses
- optimizes

time series data has EWMA

EWMA

↳ Good main one value point values to prone points

in compression me



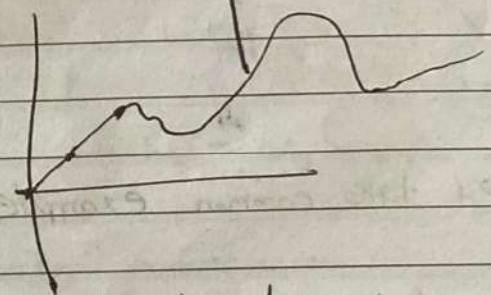
$$V_t = \beta V_{t-1} + (1-\beta) Q_t$$

let $\boxed{\beta = 0.9}$ or $\boxed{V_0 = 0}$

$$V_1 = 0.9 \times V_0 + 0.1 \times 13 = 1.3$$

$$V_2 = 0.9 \times 1.3 + 0.1 \times 17 =$$

index	temp (°C)
D1	21
D2	13
D3	17
D4	21
D5	23



$\beta \rightarrow \boxed{\frac{1}{1-\beta}}$ How many past data should depend

If $\beta = 0.9 = \frac{1}{0.1} = 10 \rightarrow$ Average of last 10 days

$\beta = 0.5$

$\frac{1}{0.5} = 2 \rightarrow$ Average of last 2 days

β is decaying factor, reduce values very fast information

1. SGD with Momentum

Why we use this?

Non convex optimization

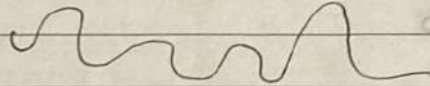
1) High curvature



2) Consistent gradient



3) Noisy gradient



→ slope very low
Speeds slow

Let take common example

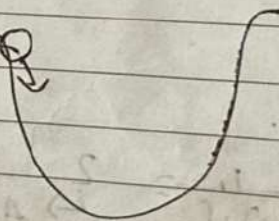
If someone ask for direction to 4 people if they point to same direction the speed will be high

A → → →

B



Acts like a ball



Momentum $m = mv$

↳ $m \rightarrow v$

previous history

weight updation = $w_n = w_0 - \eta \frac{dL}{dw_0}$

$\left[\text{let } \frac{dL}{dw_0} = g_t \right]$

normal SGD

we use velocity in SGD with "momentum"

$v_t = \beta(v_{t-1}) + (1-\beta)g_t \rightarrow \beta(v_{t-1}) + \eta g_t$
 $\hookrightarrow$ using EWMA

$\left[w_n = w_0 - \eta v_t \right] \rightarrow w_n = w_0 - v_t$

SGD with momentum

$w_n = w_0 - \eta (\beta(v_{t-1}) + (1-\beta)g_t)$

if $\beta = 0 \rightarrow$ normal SGD

$w_n = w_0 - v_t$

$w_n = w_0 - \eta g_t$

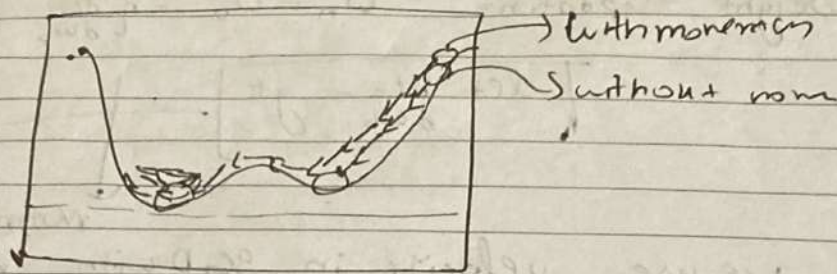
$\beta \rightarrow$ decaying factor

$0 < \beta < 1$

$\hookrightarrow$ Between 0 & 1

if $\beta = 0 \rightarrow$ normal SGD

if $\beta = 1 \rightarrow$ if only depend on past data not present data



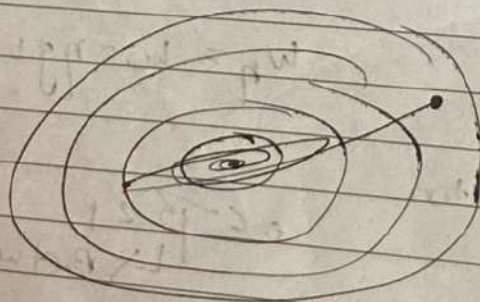
With momentum it oscillates, zig zag motion while reaching minima.

Problem →

↓ It can't become oscillation

fastest became of

Agar mein parame values ko kam importance du ∇ then oscillation kam hote hain



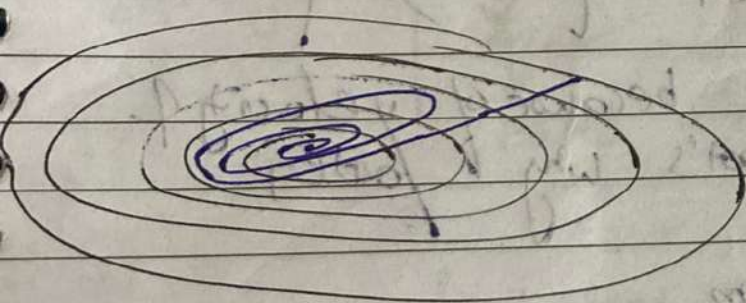
2. Nesterov Accelerated Gradient (NAG)

Non convex problem have momentum k through kaffi time + lga hai convergence mein because of oscillation
the solution of this problem is NAG

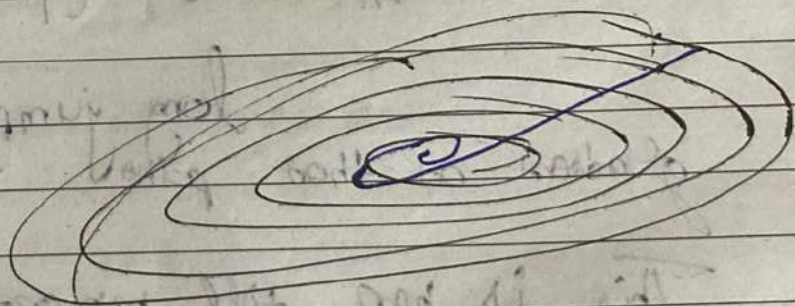
↓
oscillation ko damp karta hai NAG

↓
reduce

1/ SGD with momentum have $\beta=0.9$ it will oscillate very much, such that takes 40 epochs to reach optimal answer, whereas NAG at same decay factor $\beta=0.9$ will take 20-25 epochs only, reduces oscillation



SGD more



NAG

very less oscillation in NAG

Momentum

$$w_{t+1} = w_t - v_t$$

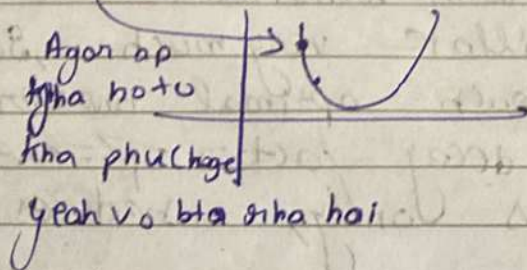
~~$$w_{t+1} = w_t - \eta v_t$$~~

$$v_t = \cancel{(1-\beta)} v_{t-1} + \eta \frac{\partial \mathcal{L}}{\partial w_t} \quad \cancel{v_t = \beta v_{t-1} + (1-\beta) \frac{\partial \mathcal{L}}{\partial w_t}}$$

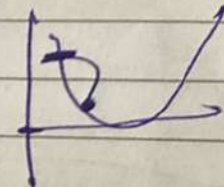
part velocity extra

$$w_{t+1} = w_t - (\beta v_{t-1} + \eta \frac{\partial \mathcal{L}}{\partial w_t}) \quad \text{GP at that point}$$

$$w_{t+1} = w_t - \eta \frac{\partial \mathcal{L}}{\partial w_t} \rightarrow \text{normal GP}$$



$$w_{t+1} = w_t - (\beta v_{t-1} + \eta \frac{\partial \mathcal{L}}{\partial w_t})$$



long jump because of velocity of gradient at that point that's why faster

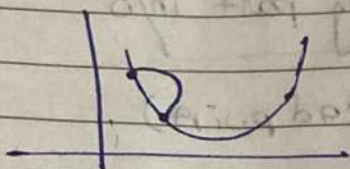
This is how till momentum

Momentum

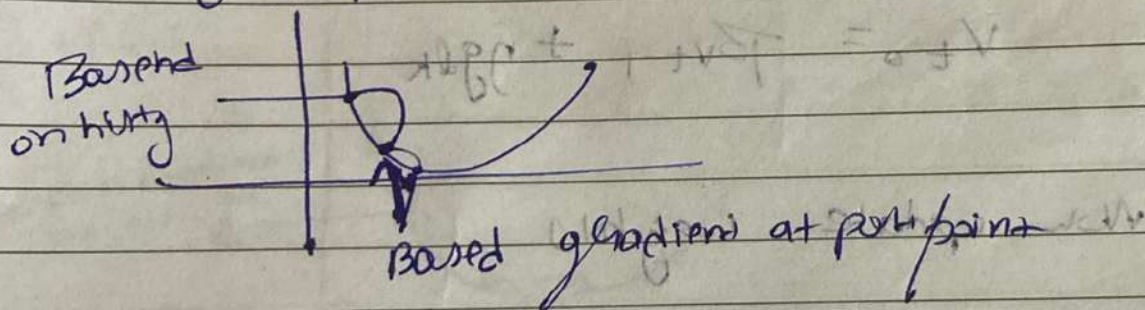
history of velocity $\oplus$ gradient at that point

ek sam calculate karne jump krte h

$$w_n = w_0 - (\beta v_{t-1} + \eta \frac{\partial L}{\partial w_0}) \rightarrow \text{ek sam jump}$$



But in NAG first jump based on history then updation based on gradient at that point to go forward or backward



lookahead term

$\hookrightarrow$ velocity k hood kha jayega

$$w_{t+1} = w_t - \beta v_{t-1}$$

$\hookrightarrow$ past momenta

$$v_{t+1} = \beta v_{t-1} + \eta \frac{\partial L}{\partial w_{t+1}} \rightarrow \text{new point}$$

$$w_{t+1} = w_t - v_t$$

Step 1 $\rightarrow$

If I am at w_t

2 $\rightarrow$ Lookahead point will be

$$w_{lk} = w_t - \beta v_{t-1}$$

$\rightarrow$ using past info

3 $\rightarrow$ calculate gradient

at new point (lookahead point)

$$g_{lk} = \frac{\partial L}{\partial w_{lookahead}} \rightarrow \frac{\partial L}{\partial w_{lk}}$$

4 $\rightarrow$ now update new velocity

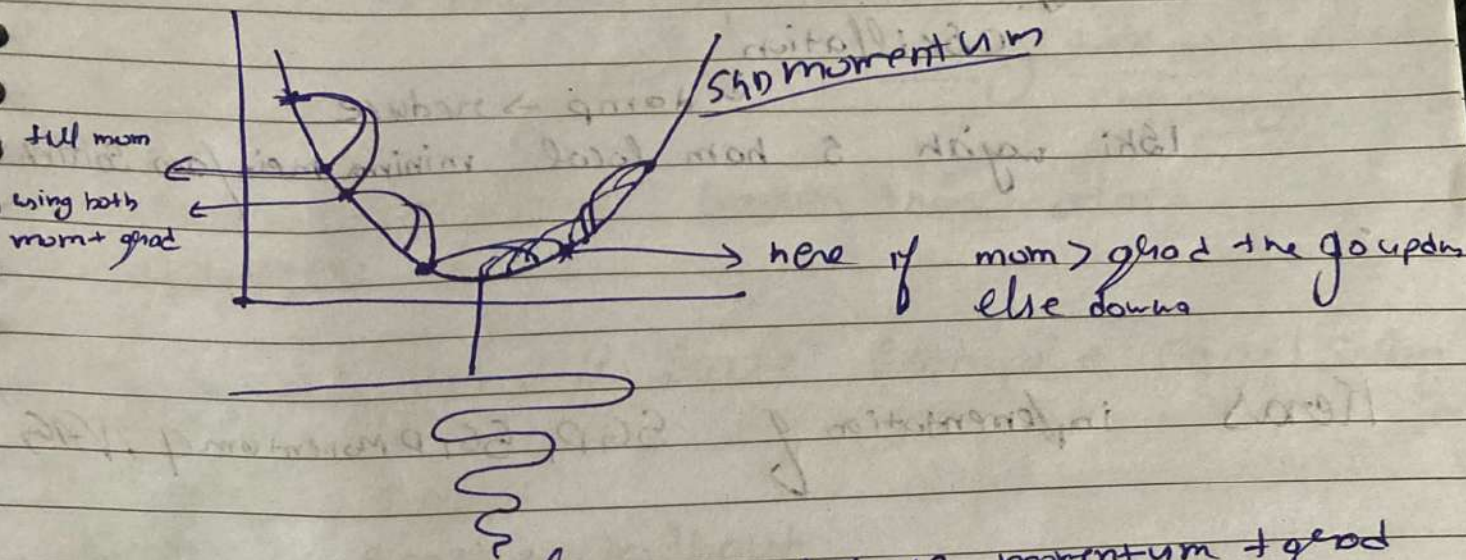
$$v_{t+1} = \beta v_t + \eta g_{lk}$$

5 $\rightarrow$ now update weights

$$w_{t+1} = w_t - v_{t+1}$$

First $\rightarrow$ lookahead point, second $\rightarrow$ gradient at that point,
third, new velocity at that point, 5 $\rightarrow$ update

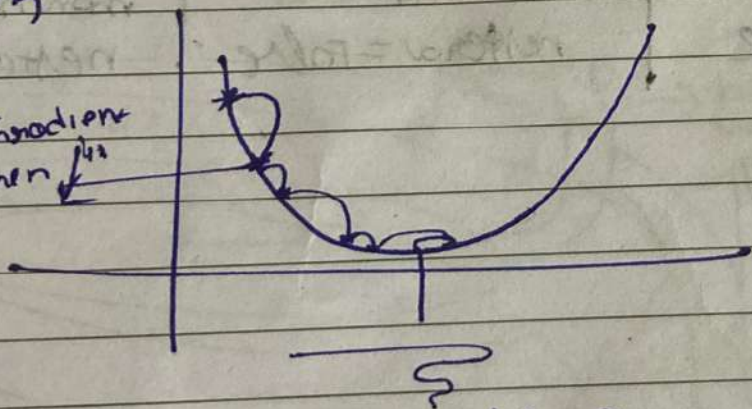
Geometric Intuition



long when because momentum + grad together give high displacement

NAG

calculate gradient then then



Short return
calculate gradient after every momentum jump

Disadvantage



Oscillation



damp → reduce

Iski wajah se ham local minima mein jaisa jhithke

Keras implementation of SGD, SGD Momentum & NAG

+ keras.optimizers.SGD

learning_rate = 0.01

SGD

momentum = 0.0

noise = False

momentum

momentum = 0.9

noise = False

NAG

momentum = 0.9

noise = True

3 → AdaGrad

↳ Adaptive Gradient

↳ Learning rate is not fixed
changes

When it performs better than other optimizers

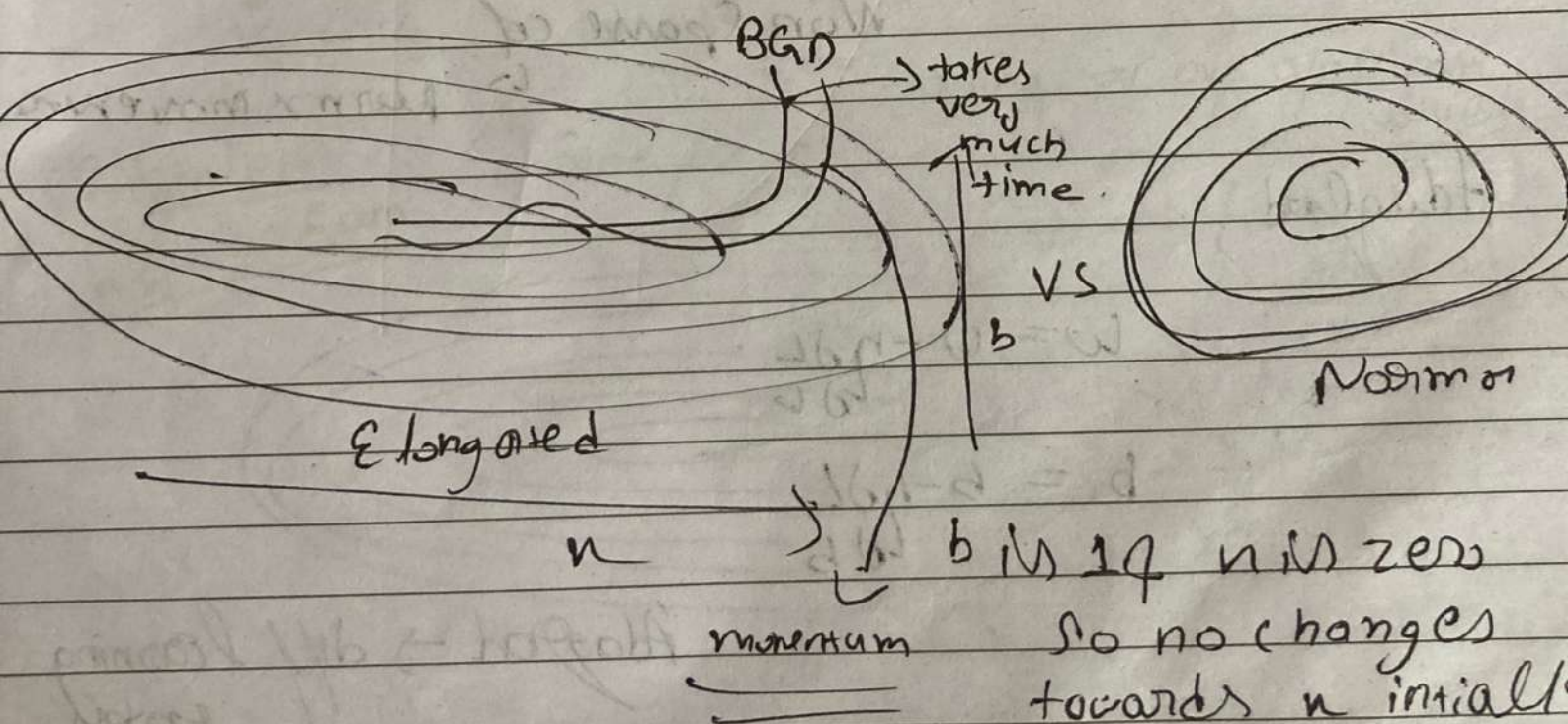
↳ Inputs on diff scale example cpm / Salary
1-10 0-5 lakhs

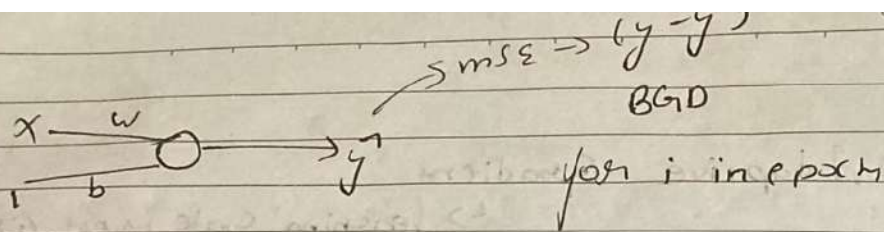
↳ having sparse in input

↳ where input contains

many zeros

we will get elongated bowl problem if we have more 0's in input





$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w}$$

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$B = B - \frac{\partial L}{\partial B}$$

$$\Rightarrow -2(y - \hat{y})u$$

most of time
it will be zero

$$\frac{\partial L}{\partial b} = -2(y - \hat{y}) \times 1$$

Big update

update will be very
small

Sparse col

$\hookrightarrow$ Small movement

Non Sparse col

$\hookrightarrow$ Large movement

Adagrad

$$w = w - \eta \frac{\partial L}{\partial w}$$

$$b = b - \eta \frac{\partial L}{\partial b}$$

Adagrad $\rightarrow$ diff learning
rates

if update is big makes it small

$$b = b - \eta \left[\frac{\partial L}{\partial b} \right]$$

if it's $\uparrow$ make it $\downarrow$

4. if derivative is small the η will be $\uparrow$
make them comparable

mathematical formula

$$w_{t+1} = w_t - \eta \frac{\partial L}{\partial w}$$

$$\sqrt{V_t + \epsilon}$$

$\epsilon \rightarrow$ prevent being zero denominator

$$V_t = V_{t-1} + (\partial w_t)^2$$

part gradient

$\downarrow$

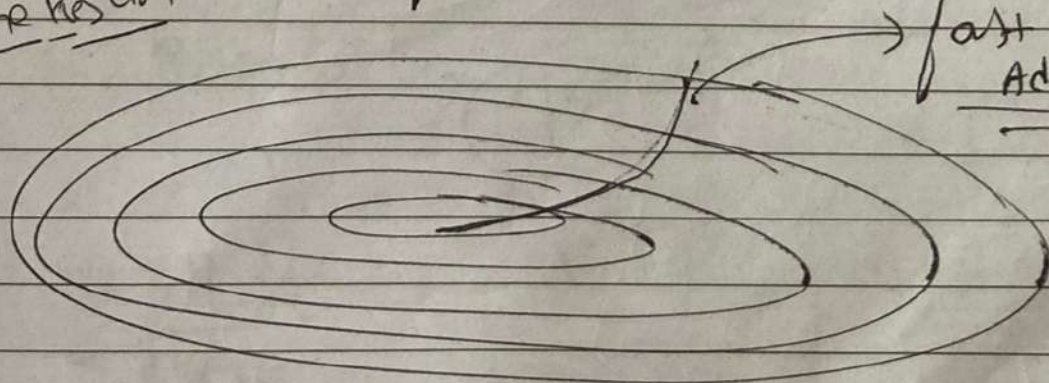
$\downarrow$

taking avg of all part values

part gradient
k square sum

Same for b

fast
Adagrad



1/15 Advantage

we don't use in neural networks because, it never reaches to solution if it reaches nearly

$\frac{\eta}{\sqrt{t+1}}$ $\rightarrow$ we are dec so it never meets & become very slow

1) RMSprop

↳ Root mean Square propagation

↳ Improvement of Adagrad

Used when sparse data in Input, contour plot get elongated where both are SGD momentum takes much time, but Adagrad solves it but never reaches to solution because we reduce learning rate by $\frac{\eta}{\sqrt{V_t + \epsilon}}$ very small
 $V_t = V_{t-1} + (\nabla w)^2$ never reaches to minima.

← RMSprop solves this problem

Adagrad

$$w_{t+1} = w_t - \eta \frac{\frac{dL}{dw}}{\sqrt{V_t + \epsilon}}$$

$$V_t = V_{t-1} + (\nabla w_t)^2$$

↑
calculating Avg of all the weights

But in RMSprop

$$\beta = 0.95$$

$$w_{t+1} = w_t - \eta \frac{\frac{dL}{dw}}{\sqrt{V_t + \epsilon}}$$

$$V_t = \beta V_{t-1} + (1 - \beta)(\nabla w_t)^2$$

↓ ↙
Taking decaying factor out
only giving high importance to recent grad than past grad

so because of that $v_t \downarrow$ get dec

so $\frac{\eta}{\sqrt{v_t + \epsilon}}$ doesn't become too small

$$v_0 = 0$$

$$v_1 = 0.95 \times 0 + 0.05 (\nabla w_1)^2$$

$$v_2 = 0.95 \times 0.05 (\nabla w_1)^2 + 0.05 (\nabla w_2)^2$$

$$v_3 = \underbrace{0.95 \times 0.95 \times 0.05 (\nabla w_1)^2}_{\text{epoch 1}} + \underbrace{0.95 \times 0.05 (\nabla w_2)^2}_{\text{epoch 2}} + \underbrace{0.05 (\nabla w_3)^2}_{\text{epoch 3}}$$

See the part gradients which we got are becoming small & heavily becoming depending on recent epoch

knowing that v_t only depends on recent where as Adagrad has dependence on both recent & past that's why v_t is more for Adagrad less for RMSprop

v_t is less so updates are more feasible in RMSprop

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \epsilon}} \frac{\partial \mathcal{L}}{\partial w}$$

$$v_t = \beta v_{t-1} + (1 - \beta) (\nabla w_t)^2$$

Disadvantage

No, it's one of the best optimization technique to optimize Neural Network

It was used mostly before Adam, if Adam not giving better result, to use RMSprop

5. Adam Adaptive Momentum Estimation



most powerful

ANW
CNW
RNN



momentum + learning decay

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t} + \epsilon} * m_t$$

no gradient here like RMSprop

momentum

$$\beta_1 = 0.9$$

$$\beta_2 = 0.99$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla w_t \rightarrow \text{momentum}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla w_t)^2 \rightarrow \text{Adagrad}$$

Bias Correction $\rightarrow m_t = 0, v_t = 0$ initially so to change it

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \leftarrow \text{epoch number}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$\eta = 0.01$

Verdict

Adam $\rightarrow$ mostly used
RMSprop $\rightarrow$
(momentum)